

SLM Field Card

Everything from the two-hour class on one page: how to size a small model, when it is the right call, when it absolutely is not, and the numbers to check before you commit engineering time.

5 tok/s Below this an interactive agent stops being tolerable on your hardware.	7–9B Floor for agentic tool use. Sub-4B is for classification and extraction only.	p^N End-to-end success compounds. Cut N before you buy parameters.
~2M/day Tokens below which self-hosting costs more in engineering than it saves.		

01 What is the task shape?

Classification, extraction, routing, format conversion → sub-4B is fine. Tool-calling agent → 7–9B floor. Planning, novel reasoning, cross-repo code → frontier. Task shape decides model size, not the other way round.

02 What is N?

Count the steps between checkpoints. Above roughly ten unverified steps, decompose before you do anything else. Four verified 5-step segments need $p = 0.979$ each; one unverified 20-step run needs 0.995 .

03 What is your volume?

Below ~2M tokens/day the engineering cost swamps the token savings. Use a hosted model and revisit when the volume is real.

04 Do you have a non-cost reason to self-host?

Sovereignty, a latency SLA, per-tenant weights, deprecation risk, air-gap. If not, a cheap hosted small model captures most of the win with none of the ops.

05 Do you have a golden set?

If not, you cannot answer any of the four questions above with evidence. Build it first: 50–100 cases from real production traffic, 20% deliberately nasty.

Two of these are separate decisions that get conflated constantly. *Small vs large* is nearly always a win on a narrow task. *Self-host vs API* is a different question, and it is almost never won on token price.

CLASS	WEIGHTS	PRACTICAL RAM	RUNS COMFORTABLY ON	USE IT FOR
1B	0.7–1.3 GB	3 GB	anything, including phones	Routing, intent, guardrails, PII screening
3B	2.0–2.2 GB	4–5 GB	an 8 GB laptop	Classification, extraction to a fixed schema, summarising bounded inputs
7–8B	4.5–6.0 GB	8 GB	a 16 GB laptop	The floor for tool-calling agents
14B	~9 GB	12–16 GB	16 GB minimum, 24 GB comfortable	Multi-tool agents, longer horizons, weaker prompts

Budget *weights* + ~1 GB runtime + KV cache. KV cache is the one everyone forgets — at 64K context it can exceed the weights. Generation speed is memory-bandwidth bound, not compute bound, which is why an M4 barely beats an M2 and loses to an M3 Pro.

Tool support is a property of the chat template, not the parameter count. A 70B with no tool template loses to a 1.7B that has one. Check the model card before you check the leaderboard.

The compounding table

END-TO-END SUCCESS = p^N

PER-STEP RELIABILITY	N=1	N=3	N=5	N=10	N=20
p = 0.50 a 4B on BFCL	50.0%	12.5%	3.1%	0.1%	0.0%
p = 0.66 a 9B on BFCL	66.0%	28.7%	12.5%	1.6%	0.0%
p = 0.75 best frontier on BFCL	75.0%	42.2%	23.7%	5.6%	0.3%
p = 0.90 with a validator	90.0%	72.9%	59.0%	34.9%	12.2%
p = 0.95 + retries	95.0%	85.7%	77.4%	59.9%	35.8%
p = 0.99 + constrained decoding, checkpoints	99.0%	97.0%	95.1%	90.4%	81.8%

Per-step p required for 90% end-to-end success: $N=5 \rightarrow 0.979 \cdot N=10 \rightarrow 0.990 \cdot N=20 \rightarrow 0.995 \cdot N=50 \rightarrow 0.998$. Nothing on any leaderboard is there — small or frontier.

Taken literally this table says nothing works, which is obviously false. The gap between it and production is exactly the engineering: retries, validators, constrained decoding, checkpoints, and short verified segments instead of long unverified ones. Reliability is an engineering outcome, not a model property — one production system took its failure rate from ~14% to 0.4% with no model change at all.

LEVEL	SIZE VS FP16	QUALITY	VERDICT
Q8_0	~50%	effectively lossless	Safe anywhere
Q5_K_M	~35%	near-lossless (Δ ppl +0.014)	Safe anywhere
Q4_K_M	~28%	Δ ppl +0.054, ~1% aggregate	The default. Start here.
Q3_K_S	~22%	HellaSwag -0.6, GSM8K -9.3	Recall only, never agents
Q2 and below	<20%	unreliable	Don't

Q4_K_M means: 4-bit nominal, *K* for k-quants (super-blocks of 256, sub-blocks of 32, each with its own quantised scale and min), *M* for medium mixed precision (attention *wv* and FFN *w2* kept at Q6_K). Net ~4.89 bits per weight — "4-bit" is never four bits, and the overhead is what buys the quality.

Damage is task-dependent, not uniform. Single-shot recall barely moves as you quantise; chained reasoning falls off a cliff. Tool-argument construction and multi-step arithmetic are chained reasoning. And if a QAT checkpoint exists for your model, take it — it cuts the quality drop roughly in half, for free.

the whole self-hosting question in one line

\$ per 1M tokens = (GPU \$/hr ÷ throughput tok/s) × 1e6 ÷ 3600 ÷ utilisation

Utilisation is the term everyone omits and it swings the answer 5×: same H100, same model — \$0.20/1M at full load, \$1.01/1M at 20%, \$4.03/1M at 5%. Your idle GPU is the biggest line item in a self-hosted quote and it never appears in the vendor's comparison table.

DISPLACING	BLENDED \$/1M	BREAK-EVEN ON ONE 4090 RIG	PER DAY
a frontier model (\$5 / \$25)	\$9.00	~57M tok/mo	1.9M
a mid-tier model (\$2 / \$10)	\$3.60	~146M tok/mo	4.9M
a cheap fast model (\$1 / \$5)	\$1.80	~305M tok/mo	10M
a cheap <i>hosted small</i> model	\$0.40	~2,000M tok/mo	67M

Assumes agent traffic at 80% input / 20% output and a ~\$504/month rig. Then add the real total: hardware \$1,000 + power and colocation \$831 + 25% of one engineer \$4,000 ≈ \$5,931/month. *Staffing is 6–30× the hardware* — and that total lands at roughly the price of the hosted API it was meant to beat.

What actually justifies self-hosting: regulation (EU AI Act high-risk obligations took effect 2 August 2026), data sovereignty, a latency SLA where 15s → 0.15s is a different product, per-tenant fine-tuned weights via multi-LoRA, and freedom from silent vendor deprecation. Not token price.

● Horizon beyond ~10 unverified steps The compounding math is unforgiving. Decompose or escalate.	● Broad world knowledge required You cannot fine-tune in facts the model does not have.
● Code generation across a large repo Needs long context and long horizon at the same time.	● Ambiguous or novel reasoning Fine-tuning specialises; it does not generalise to the case you did not anticipate.
● Very long context Degradation hits everyone, but small models start lower and hit the wall sooner.	● Low volume Below ~2M tokens/day the engineering cost swamps the savings.
● A single failure is expensive Irreversible actions, money movement, customer commitments. Even frontier models roughly halve from pass@1 to pass^8.	● Multilingual breadth Capability concentrates in high-resource languages as parameters shrink.
● Safety-critical generation Note the asymmetry: small models are excellent at safety <i>classification</i>, weaker at safety <i>alignment</i> under pressure.	● You have no golden set You will not be able to tell whether the swap was safe. This is how SLM projects actually die.

PATTERN	WHAT IT IS	EVIDENCE
Cascade	Rules → small model → escalate on a <i>validator's verdict</i> , never on the model's self-reported confidence.	1.5M checks/mo: 98% by logistic regression, 2% by a tuned 8B. 80–82% → 90% accuracy, 15s → 0.15s, \$12k/mo → <\$800.
Heterogeneous	SLM for the mechanical high-frequency work; frontier model for planning and error recovery.	Agent frameworks estimate 40–70% of calls are replaceable.
Frontier as offline teacher	The big model manufactures training data; the small model runs in the hot path and the big one never appears at inference.	An 8B edit-prediction model distilled from a frontier teacher: +30% acceptance rate.
Guardrails as SLMs	Safety and PII classification is a small-model job.	Across 14 guard models 110M–20B: correlation of size with detection $r=0.21$, $p=0.48$. A 4B had 3.4× the recall of a 20B.

Evaluation

- 01
- Score per tag, never just the average.** If 90% of your cases are happy-path, your average looks great while every hard case quietly fails.
- 02
- Source cases from production failures, not imagination.** Instrument for thumbs-down, abandonment and escalation. Triage weekly. Prune cases testing behaviour you deliberately removed.
- 03
- Size it honestly.** 50–100 cases for one feature. 150–300 for a maturing product. 400+ for a regulated flow. 15–25% deliberately nasty.
- 04
- Validate the judge before you trust it.** Across 21 judge models, chance-corrected agreement ran 34–41 points below raw agreement — the best scored 84.9% raw but 51.1% kappa. Give it a specific rubric, randomise A/B order, check against ~50 human labels. Several judges were perfectly reproducible while carrying severe position bias: reproducibility is not validity.
- 05
- ±1 point on a 200-item set is noise.** Quantised models sometimes "beat" fp16 on a benchmark. That is variance, not a discovery.

Tooling: *promptfoo* for a model bake-off · *deepeval* for CI gates · *lm-evaluation-harness* for academic benchmarks only · *ragas* for RAG.


```
# the single most important line: Ollama defaults to a 4K context on
# machines under 24GB VRAM, which silently truncates your tool schemas
OLLAMA_CONTEXT_LENGTH=32000 ollama serve

ollama pull granite4.2:3b          # 2.2 GB  workhorse, tools + JSON schema
ollama pull granite4.2:8b          # 5.3 GB  escalation target
ollama pull lfm2.5-thinking:1.2b   # 731 MB  runs on almost anything

# OpenAI-compatible endpoint. api_key is required but ignored.
http://localhost:11434/v1/        # tool_choice is NOT supported here

# native API: structured output via a full JSON Schema in "format",
# tools via "tools", and real timing counters in the response
POST http://localhost:11434/api/chat

# fine-tuning: r=32, alpha=64, lr=2e-4, 1-3 epochs, all seven projections
# q,k,v,o,gate,up,down -- attention-only underperforms
# QLoRA VRAM: 3B in 3.5GB, 8B in 6GB. Both fit a free Colab T4.
# Data: 100-300 examples per class for routing, 200-500 for extraction.
```

Model tags move fast — verify the exact tag and file size before you rely on one. Note that a bare tag is often Q8_0, not Q4; ask for -q4_K_M explicitly when size matters.

01	Log everything Every non-user-facing call: prompt, response, tool calls, latency. Encrypted. Without traces there is no project.
02	Curate Strip PII and PHI. Target 10k–100k examples.
03	Cluster Unsupervised clustering over prompts and operations. The clusters are your specialisation candidates.
04	Select a model Capability, licence, deployment footprint — in that order.
05	Fine-tune LoRA or QLoRA, or distil from the model you are replacing.
06	Iterate Retrain as usage drifts.

One action item: turn on structured trace logging this week. Everything else on this card is downstream of it.

Sources for every number here: [NVIDIA Small Language Models are the Future of Agentic AI \(arXiv 2506.02153\)](#) · [The Illusion of Diminishing Returns \(arXiv 2509.09677\)](#) · [τ-bench \(arXiv 2406.12045\)](#) · [Benchmarking Open-Source Safety Guard Models \(arXiv 2605.28830\)](#) · [Berkeley function-calling leaderboard](#) · [llama.cpp quantisation benchmarks](#) · [Unsloth LoRA guide](#) · [Ollama documentation](#).

Model tags, API prices and leaderboard positions change fast. Re-verify anything you are about to put in a business case.